```
File.open(name = gets.chomp)
```

Ruby has two basic forms of assignment. The first assigns an object reference to a variable or constant. This form of assignment is hardwired into the language:

```
instrument = "piano"
MIDDLE_A = 440
```

The second form of assignment involves having an object attribute or element reference on the left side. These forms are special, because they are implemented by calling methods in the lvalues, which means you can override them.

We've already seen how to define a writable object attribute. Simply define a method name ending in an equals sign. This method receives as its parameter the assignment's rvalue. We've also seen that you can define [] as a method:

```
class ProjectList
  def initialize
    @projects = []
  end

  def projects=(list)
    @projects = list.map(&:upcase)  # store list of names in uppercase
  end

  def [](offset)
    @projects[offset]
  end
end

list = ProjectList.new
list.projects = %w{strip sand prime sand paint sand paint rub paint}
list[3]   # => "SAND"
list[4]   # => "PAINT"
```

As this example shows, these attribute-setting methods don't have to correspond with internal instance variables, and you don't need an attribute reader for every attribute writer (or vice versa).

The value of the assignment is *always* the value of the parameter; the return value of the method is discarded. In the code that follows, result will be set to 2, even though the attribute setter actually returns 99.

```
class Test
  def val=(val)
    @val = val
    return 99
  end
end

t = Test.new
result = (t.val = 2)
result    # => 2
```

You can also flip the order of assignments in Ruby using the => operator, which is sometimes called "rightward assignment", and is a special case of Pattern Matching, on page 162.

```
2 => x
```